

EXPERIMENT 9

Student Name: Pranav Sood

UID: 24BDA70188

Branch: AIT-CSE

Section/Group: 24AIT_KRG-G2

Semester: 5

Subject Name: ADBMS

Subject Code: 24CSP-333

Experiment 9 -- To study and implement Procedures

Aim : To study and learn the about the Procedures, parameterised and non-parameterised procedures.

Question :9 Fetch all the employees with respect to the gender passed to the stored procedure as IP Parameter.

SQL Query:

```
DROP TABLE IF EXISTS employee_info;
```

```
CREATE TABLE employee_info (
```

```
    id SERIAL PRIMARY KEY,
```

```
    name VARCHAR(50) NOT NULL,
```

```
    gender VARCHAR(10) NOT NULL,
```

```
    salary NUMERIC(10,2) NOT NULL,
```

```
    city VARCHAR(50) NOT NULL
```

```
);
```

```
INSERT INTO employee_info (name, gender, salary, city)
```

```
VALUES
```

```
('Alok', 'Male', 50000.00, 'Delhi'),
```

```
('Priya', 'Male', 60000.00, 'Mumbai'),
```

```
('Rajesh', 'Female', 45000.00, 'Bangalore'),
```

EXPERIMENT 9

```
('Sneha', 'Male', 55000.00, 'Chennai'),
('Anil', 'Male', 52000.00, 'Hyderabad'),
('Sunita', 'Female', 48000.00, 'Kolkata'),
('Vijay', 'Male', 47000.00, 'Pune'),
('Ritu', 'Male', 62000.00, 'Ahmedabad'),
('Amit', 'Female', 51000.00, 'Jaipur');
```

```
CREATE OR REPLACE PROCEDURE sp_get_employees_by_gender(
    IN p_gender VARCHAR(50),
    OUT p_employee_count INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT COUNT(id)
    INTO p_employee_count
    FROM employee_info
    WHERE gender = p_gender;
    RAISE NOTICE 'Total employees with gender %: %', p_gender, p_employee_count;
END;
$$;
```

```
CALL sp_get_employees_by_gender('Male', NULL);
```

	p_employee_count integer	
1		12

EXPERIMENT 9

Question 9: Part II) An e-commerce platform needs an automated backend process to handle customer purchases. Create a stored procedure that accepts a product name and a desired quantity

to process an order. The procedure must verify if sufficient stock exists; if it does, it should dynamically log a new entry into the sales registry, calculate the total order price, deduct the purchased units from the remaining inventory, and increment the overall quantity sold. If stock is insufficient, it should halt execution and flag an appropriate notification message.

SQL Query

```
CREATE OR REPLACE PROCEDURE pr_buy_products(  
    IN p_product_name VARCHAR,  
    IN p_quantity INT  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_product_code VARCHAR(20);  
    v_price FLOAT;  
    v_count INT;  
BEGIN  
  
    -- Step 1: Check if product exists and has enough quantity  
    SELECT COUNT(*)  
    INTO v_count  
    FROM products  
    WHERE product_name = p_product_name
```

EXPERIMENT 9

AND quantity_remaining >= p_quantity;

-- Step 2: If sufficient stock

IF v_count > 0 THEN

-- Fetch product code and price

SELECT product_code, price

INTO v_product_code, v_price

FROM products

WHERE product_name = p_product_name;

-- Insert a new record into the sales table

INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)

VALUES (CURRENT_DATE, v_product_code, p_quantity, (v_price * p_quantity));

-- Update stock details

UPDATE products

SET quantity_remaining = quantity_remaining - p_quantity,

quantity_sold = quantity_sold + p_quantity

WHERE product_code = v_product_code;

-- Confirmation message

RAISE NOTICE 'PRODUCT SOLD..! Order placed successfully for % unit(s) of %.', p_quantity,
p_product_name;

ELSE

-- Step 3: If stock is insufficient

EXPERIMENT 9

RAISE NOTICE 'INSUFFICIENT QUANTITY..! Order cannot be processed for % unit(s) of %.',
p_quantity, p_product_name;

END IF;

END;

\$\$;

CALL pr_buy_products ('MacBook Pro 14"', 1);

```
NOTICE:  PRODUCT SOLD..! Order placed successfully for 1 unit(s) of MacBook Pro 14".  
CALL
```

```
Query returned successfully in 37 msec.
```

Learning Outcome

After completing this experiment, the learner will be able to:

1. **Understand and implement** Procedures.
2. Parameterised Procure to apply business logic.
- 3.